Intro to DevOps

Practice Questions — Complete Answers

LO4 • LO5 • LO6 — Simple English Explanations with Code Examples

LO4 — Accelerated Delivery Using Containers

Q1: Create a Deployment named web running nginx:1.25 with 3 replicas, then export its YAML to a file.

→ Use kubectl create deployment with --replicas flag, then use -o yaml and redirect to a file.

```
kubectl create deployment web --image=nginx:1.25 --replicas=3
kubectl get deployment web -o yaml > web-deployment.yaml
```

✅ The first command creates the deployment. The second exports it to a file you can edit later.

Q2: Enable pulling images from DockerHub as authenticated user. What resource do you need?

→ You need to create an image-pull Secret. This stores your DockerHub credentials so Kubernetes can pull private images without hitting rate limits.

```
kubectl create secret docker-registry dockerhub-secret \
  --docker-username=YOUR_USERNAME \
  --docker-password=YOUR_PASSWORD \
  --docker-email=YOUR_EMAIL
```

Then reference it in your deployment YAML:

```
spec:
  imagePullSecrets:
  - name: dockerhub-secret
```

✅ Without this secret, DockerHub limits unauthenticated pulls to ~100 per 6 hours per IP.

Q3: Scale web from 3 to 5 replicas two different ways — kubectl scale and by editing the manifest.

Way 1 — kubectl scale (fast, imperative):

```
kubectl scale deployment web --replicas=5
```

Way 2 — edit the manifest (declarative):

```
kubectl edit deployment web
```

Find the line 'replicas: 3' and change it to 'replicas: 5', then save and exit (:wq in vim)

Show the resulting ReplicaSet:

```
kubectl get replicaset
```

✅ You'll see the old ReplicaSet with 0 pods and the new one with 5 pods. Kubernetes creates a new ReplicaSet for each change.

Q4: Perform a rolling update from nginx:1.25 to nginx:1.27 and watch it with rollout status.

→ A rolling update replaces old pods one by one — no downtime. The app stays available throughout.

```
kubectl set image deployment/web nginx=nginx:1.27
kubectl rollout status deployment/web
```

You'll see output like: 'Waiting for deployment to finish: 1 out of 3 new replicas have been updated...'

✅ Rolling update = old pods die one by one while new pods start. Traffic always goes to working pods.

Q5: View rollout history and roll back to previous revision. Explain CHANGE-CAUSE.

```
kubectl rollout history deployment/web
```

Output shows revisions. CHANGE-CAUSE is a note explaining WHY that change was made.

To populate CHANGE-CAUSE, add --record flag (deprecated) or set the annotation manually:

```
kubectl annotate deployment/web kubernetes.io/change-cause='Updated to nginx
1.27'
```

Roll back to previous version:

```
kubectl rollout undo deployment/web
```

Roll back to a specific revision:

```
kubectl rollout undo deployment/web --to-revision=1
```

✅ CHANGE-CAUSE is just a text note. It helps you remember what each revision was for.

Q6: Set RollingUpdate strategy with maxSurge:1 and maxUnavailable:0. Explain the effect.

→ maxSurge:1 = allow 1 EXTRA pod during update. maxUnavailable:0 = never take a pod down before a new one is ready. This means zero downtime but you briefly run N+1 pods.

```
kubectl patch deployment web -p '{
  "spec": {
    "strategy": {
      "type": "RollingUpdate",
      "rollingUpdate": {
        "maxSurge": 1,
        "maxUnavailable": 0
      }
    }
  }
}'
```

✅ With maxUnavailable:0, you always have at least 3 pods running. With maxSurge:1, temporarily 4 pods run. No requests are dropped.

Q7: Switch strategy to Recreate and describe when it's required.

→ Recreate kills ALL old pods first, then starts new ones. This causes downtime but is needed when old and new versions CANNOT run at the same time.

Example scenario: A database migration that changes the schema. If old pods are still running with the old schema while new pods expect the new schema, both will fail. You must stop everything, migrate, then start new version.

```
kubectl patch deployment web -p '{"spec":{"strategy":{"type":"Recreate"}}}'
```

⚠️ Recreate = downtime. Only use it when the old and new versions are incompatible with each other.

Q8: Add CPU/memory requests and limits to a Deployment and verify in running pod.

→ Requests = minimum guaranteed. Limits = maximum allowed. Without these, a pod can use all node resources.

```
kubectl set resources deployment/web \
  --requests=cpu=100m,memory=128Mi \
  --limits=cpu=500m,memory=256Mi
```

Verify it worked:

```
kubectl get pod -l app=web -o jsonpath='{.items[0].spec.containers[0].resources}'
```

✅ 100m CPU = 0.1 of a CPU core. 128Mi = 128 megabytes of RAM.

Q9: Set revisionHistoryLimit:3 and explain its effect on rollbacks.

→ This setting controls how many old ReplicaSets (previous versions) Kubernetes keeps. With limit 3, you can only roll back 3 versions. Older ReplicaSets are deleted to save space.

```
kubectl patch deployment web -p '{"spec":{"revisionHistoryLimit":3}}'
```

✅ Default is 10. Set lower to save memory. But you lose the ability to roll back further.

Q10: Set image to non-existent tag, observe rollout gets stuck, explain why old pods keep serving.

```
kubectl set image deployment/web nginx=nginx:doesnotexist999
kubectl rollout status deployment/web
```

The rollout gets stuck because the new image can't be pulled. You'll see ImagePullBackOff on new pods.

→ Old pods keep running because Kubernetes only kills old pods AFTER new ones are healthy. Since new pods never become healthy, old pods are never killed. Your users see no disruption.

Roll back to fix it:

```
kubectl rollout undo deployment/web
```

✅ This is the beauty of RollingUpdate — bad deployments don't take down your app.

Q11: Use label selector to list pods of one Deployment. Explain Deployment → ReplicaSet → Pod relationship.

```
kubectl get pods -l app=web
```

→ Deployment manages ReplicaSets. ReplicaSet manages Pods. Labels connect them all.

- Deployment has a selector (e.g. app=web)
- It creates a ReplicaSet with the same selector
- ReplicaSet creates Pods with the matching label
- If you update the Deployment, it creates a NEW ReplicaSet

```
kubectl get replicaset -l app=web
kubectl get pods -l app=web
```

Q12: Expose a Deployment with kubectl expose. Explain what was created and how selector was derived.

```
kubectl expose deployment web --port=80 --target-port=80
```

→ This creates a Service. The Service's selector is automatically copied from the Deployment's pod template labels (app=web). The Service uses this selector to find which pods to send traffic to.

```
kubectl get service web
kubectl describe service web   # look at Selector field
```

Q13: Add a sidecar container to a Deployment and explain how containers share network and volumes.

→ All containers in a pod share the same network namespace (same IP, same ports) and can share volumes. A sidecar is a helper container running alongside the main one.

```
kubectl edit deployment web
```

In the containers section, add:

```
containers:
- name: nginx
  image: nginx:1.25
- name: log-sidecar        # second container
  image: busybox
  command: ['sh', '-c', 'tail -f /var/log/nginx/access.log']
  volumeMounts:
  - name: logs
    mountPath: /var/log/nginx
```

✅ Both containers share the same localhost. If nginx listens on port 80, the sidecar can reach it at localhost:80.

Q14: Write Deployment manifest for httpd:2.4 with 2 replicas, named port, and nodeSelector. Explain Pending.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpd-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: httpd-app
  template:
    metadata:
      labels:
        app: httpd-app
    spec:
      nodeSelector:
        disktype: ssd      # node must have this label
      containers:
      - name: httpd
        image: httpd:2.4
        ports:
        - name: http
          containerPort: 80
```

→ If no node has the label disktype=ssd, pods stay in Pending forever because the scheduler can't find a matching node.

Q15: Create bare Pod running busybox. Explain what happens when you delete it vs a Deployment-managed pod.

```
kubectl run bare-pod --image=busybox --restart=Never -- sleep 3600
```

→ Bare pod = deleted permanently when you run kubectl delete pod bare-pod. Nobody recreates it.

→ Deployment-managed pod = when you delete it, the ReplicaSet immediately creates a replacement. The deployment keeps the desired number of pods running.

```
kubectl delete pod bare-pod          # gone forever
kubectl delete pod web-abc123        # ReplicaSet creates a new one immediately
```

Q16: Write StatefulSet for redis:7 with 3 replicas plus headless Service. Show pod names and DNS.

```
# First create the headless service
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  clusterIP: None    # this makes it headless
  selector:
    app: redis
  ports:
  - port: 6379
---
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: redis
spec:
  serviceName: redis
  replicas: 3
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
      - name: redis
        image: redis:7
        ports:
        - containerPort: 6379
```

Pod names are predictable:

- redis-0
- redis-1
- redis-2

DNS records for each pod:

- redis-0.redis.default.svc.cluster.local
- redis-1.redis.default.svc.cluster.local
- redis-2.redis.default.svc.cluster.local

Q17: Create a DaemonSet running busybox. Explain why exactly one pod runs per node.

→ A DaemonSet is designed to run exactly ONE copy on EVERY node. When a new node joins the cluster, the DaemonSet automatically puts a pod on it. Used for monitoring agents, log collectors, etc.

```
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: busybox-agent
spec:
  selector:
    matchLabels:
      app: busybox-agent
  template:
    metadata:
      labels:
        app: busybox-agent
    spec:
      containers:
      - name: agent
        image: busybox
        command: ['sh', '-c', 'while true; do echo running; sleep 60; done']
```

Q18: Create a Job using busybox that computes once and completes. Show COMPLETIONS and logs.

```
kubectl create job compute-job \
  --image=busybox \
  -- /bin/sh -c 'echo Result: $((2+2))'
kubectl get jobs
```

You'll see COMPLETIONS: 1/1 when done

```
kubectl logs job/compute-job
```

Output: Result: 4

✅ Jobs are for one-time tasks. When the container exits successfully, the Job is marked Complete.

Q19: Create a CronJob that prints date every minute. Show how to suspend it and list spawned Jobs.

```
kubectl create cronjob date-printer \
  --image=busybox \
  --schedule='*/1 * * * *' \
  -- date
```

Suspend it (stops future runs, doesn't delete current):

```
kubectl patch cronjob date-printer -p '{"spec":{"suspend":true}}'
```

List Jobs it has spawned:

```
kubectl get jobs -l job-name=date-printer
```

Q20: Manually run a Job from an existing CronJob to test on demand.

```
kubectl create job test-run --from=cronjob/date-printer
kubectl logs job/test-run
```

✅ This runs the job immediately without waiting for the schedule. Great for testing.

Q21: Show that StatefulSet creates/terminates pods in ORDER. Contrast with Deployment.

→ StatefulSet starts pods in order (0, 1, 2) and waits for each to be ready before starting the next. It terminates in REVERSE order (2, 1, 0). Deployment starts and stops all pods at the same time with no guaranteed order.

```
kubectl get pods -w   # watch pods being created one by one
```

StatefulSet: redis-0 starts → becomes Ready → redis-1 starts → becomes Ready → redis-2 starts

Deployment: all 3 pods start at roughly the same time

✅ Order matters for databases — the primary node (redis-0) must be ready before replicas (redis-1, redis-2) connect to it.

Q22: Create multi-container Pod sharing emptyDir — one writes, one reads. Prove it's shared.

```
apiVersion: v1
kind: Pod
metadata:
  name: shared-volume-pod
spec:
  volumes:
  - name: shared
    emptyDir: {}
  containers:
  - name: writer
    image: busybox
    command: ['sh','-c','echo hello > /shared/message.txt && sleep 3600']
    volumeMounts:
    - name: shared
      mountPath: /shared
  - name: reader
    image: busybox
    command: ['sh','-c','sleep 5 && cat /shared/message.txt && sleep 3600']
    volumeMounts:
    - name: shared
      mountPath: /shared
```

Prove it's shared:

```
kubectl logs shared-volume-pod -c reader
```

Output: hello

Q23: List the StorageClasses, PVs and PVCs.

```
kubectl get storageclass
kubectl get pv
kubectl get pvc
kubectl get pvc -A   # across all namespaces
```

Q24: Mount ConfigMap as volume so each key becomes a file. Verify contents inside pod.

```
kubectl create configmap app-config \
  --from-literal=database.host=mysql \
  --from-literal=database.port=3306
```

Create a pod that mounts it:

```
apiVersion: v1
kind: Pod
metadata:
  name: config-pod
spec:
  volumes:
  - name: config
    configMap:
      name: app-config
  containers:
  - name: app
    image: busybox
    command: ['sleep', '3600']
    volumeMounts:
    - name: config
      mountPath: /etc/config
```

Verify inside pod:

```
kubectl exec config-pod -- ls /etc/config
kubectl exec config-pod -- cat /etc/config/database.host
```

Output: mysql

Q25: Mount a single ConfigMap key to a specific path using subPath. Explain when needed and update caveat.

→ subPath mounts just ONE key from a ConfigMap to a specific file path, without replacing the whole directory. Without subPath, mounting a ConfigMap replaces the entire directory.

```
volumeMounts:
- name: config
  mountPath: /etc/nginx/nginx.conf
  subPath: nginx.conf    # only mount this one key
```

⚠️ Caveat: When you update the ConfigMap, files mounted with subPath do NOT auto-update. The pod must be restarted to see changes. Files mounted WITHOUT subPath do auto-update (eventually).

Q26: Mount a Secret as volume and verify files contain decoded values with restrictive permissions.

```
kubectl create secret generic db-creds \
  --from-literal=username=admin \
  --from-literal=password=secret123
apiVersion: v1
kind: Pod
metadata:
  name: secret-pod
spec:
  volumes:
  - name: creds
    secret:
      secretName: db-creds
      defaultMode: 0400   # read-only, owner only
```

```
  containers:
  - name: app
    image: busybox
    command: ['sleep','3600']
    volumeMounts:
    - name: creds
      mountPath: /etc/secrets
```

Verify decoded values and permissions:

```
kubectl exec secret-pod -- cat /etc/secrets/username
kubectl exec secret-pod -- ls -la /etc/secrets
```

Q27: Show StatefulSet creates one PVC per replica. Explain what happens to PVCs when StatefulSet is deleted.

→ StatefulSet creates one PVC per pod automatically. When you DELETE the StatefulSet, the PVCs are NOT deleted — data is preserved. You must delete PVCs manually if you want to remove the data.

```
kubectl get pvc   # see one per replica: data-redis-0, data-redis-1, data-redis-2
kubectl delete statefulset redis
kubectl get pvc   # PVCs still exist!
```

✅ This is intentional — Kubernetes protects your data. You decide when to delete storage.

Q28: Use initContainer to pre-populate data into shared volume before main container reads it.

→ An initContainer runs BEFORE the main container starts. It can set up files, wait for services, or do any setup work.

```
apiVersion: v1
kind: Pod
metadata:
  name: init-pod
spec:
  volumes:
  - name: shared
    emptyDir: {}
  initContainers:
  - name: setup
    image: busybox
    command: ['sh','-c','echo pre-loaded data > /data/file.txt']
    volumeMounts:
    - name: shared
      mountPath: /data
  containers:
  - name: main
    image: busybox
    command: ['sh','-c','cat /data/file.txt && sleep 3600']
    volumeMounts:
    - name: shared
      mountPath: /data
```

Q29: Set readOnly:true on a volume mount and prove writes are rejected.

```
volumeMounts:
- name: config
  mountPath: /etc/config
  readOnly: true
```

Test it:

```
kubectl exec POD_NAME -- touch /etc/config/test
```

Output: touch: /etc/config/test: Read-only file system

Q30: Set sizeLimit on emptyDir. Explain what happens if container exceeds it.

```
volumes:
- name: temp
  emptyDir:
    sizeLimit: 100Mi
```

→ If the container writes more than 100Mi to this volume, Kubernetes evicts (kills) the pod. The pod gets restarted by the Deployment/ReplicaSet.

Q31: Create generic Secret from literals. Show values are base64-encoded, not encrypted.

```
kubectl create secret generic db-secret \
  --from-literal=username=admin \
  --from-literal=password=mypassword
```

Show the stored values are base64:

```
kubectl get secret db-secret -o yaml
```

You'll see: username: YWRtaW4=

Decode it:

```
echo YWRtaW4= | base64 --decode
```

Output: admin

⚠️ Secrets are NOT encrypted — just base64 encoded. Anyone with access to the secret can decode it instantly. For real encryption, you need etcd encryption or a secrets manager like Vault.

Q32: Create Secret from files holding cert and key. Identify resulting keys.

```
kubectl create secret generic tls-files \
  --from-file=cert.pem=/path/to/cert.pem \
  --from-file=key.pem=/path/to/key.pem
```

→ The resulting keys inside the secret are exactly the filenames: cert.pem and key.pem

```
kubectl describe secret tls-files
```

Q33: Create kubernetes.io/tls typed Secret from cert/key. Where is it consumed?

```
kubectl create secret tls my-tls-secret \
  --cert=/path/to/cert.pem \
  --key=/path/to/key.pem
```

→ TLS secrets are consumed by Ingress controllers and Routes to terminate HTTPS connections. The ingress/route reads the cert and key to handle SSL/TLS encryption on behalf of your app.

Q34: Mount a Secret as volume. Security trade-offs vs env vars.

Volume mount advantages:

- Values are files — easier for apps that read config from files (like nginx)
- Values auto-update when Secret changes (without pod restart)
- You can set restrictive file permissions (chmod 0400)

Env var advantages:

- Simpler to access in code (process.env.PASSWORD)
- No file path needed

Env var disadvantages:

- More likely to accidentally leak in logs
- Do NOT auto-update when Secret changes
- Visible in 'kubectl describe pod'

Q35: Use envFrom with secretRef to load every key of a Secret as env vars at once.

```
spec:
  containers:
  - name: app
    image: myapp
    envFrom:
    - secretRef:
        name: db-secret
```

→ Every key in db-secret becomes an environment variable in the container. If secret has username and password keys, the container gets USERNAME and PASSWORD env vars.

Q36: Create docker-registry Secret and reference via imagePullSecrets. When is it required?

```
kubectl create secret docker-registry my-registry-secret \
  --docker-server=registry.example.com \
  --docker-username=myuser \
  --docker-password=mypassword
```

Reference in pod spec:

```
spec:
  imagePullSecrets:
  - name: my-registry-secret
  containers:
  - name: app
    image: registry.example.com/myapp:latest
```

→ Required when pulling from private registries (Quay.io private repos, AWS ECR, Google Container Registry, etc.) or when you've hit DockerHub's rate limit for anonymous pulls.

Q37: Create Secret with several keys and selectively mount only one into a pod.

```
kubectl create secret generic multi-secret \
  --from-literal=db_user=admin \
  --from-literal=db_pass=secret \
  --from-literal=api_key=xyz123
```

Mount only db_pass:

```
volumes:
- name: pass-only
```

```
  secret:
    secretName: multi-secret
    items:
    - key: db_pass
      path: password.txt    # this is the filename in the pod
```

Q38: Create ClusterIP Service and resolve it by DNS from another pod.

```
kubectl expose deployment web --name=web-svc --port=80
```

From another pod, test DNS resolution:

```
kubectl run dns-test --rm -it --image=busybox -- sh
# Inside the pod:
nslookup web-svc.default.svc.cluster.local
wget -qO- http://web-svc
```

Q39: Create NodePort Service and reach it.

```
kubectl expose deployment web --type=NodePort --port=80
kubectl get service web   # note the NodePort (e.g. 30080)
# On minikube:
minikube service web --url
# Or manually:
curl $(minikube ip):30080
```

Q40: Create LoadBalancer Service. Explain why EXTERNAL-IP is pending on minikube.

```
kubectl expose deployment web --type=LoadBalancer --port=80
kubectl get service web
```

→ On minikube, EXTERNAL-IP shows <pending> because LoadBalancer needs a cloud provider (AWS, GCP, Azure) to provision a real external IP address. Minikube runs locally and has no cloud provider.

Fix on minikube:

```
minikube tunnel
```

This creates a network tunnel and assigns a local IP to the service.

Q41: Create headless Service for StatefulSet. Show it returns per-pod A records.

→ A regular service returns ONE IP (virtual IP). A headless service (clusterIP: None) returns the IP addresses of ALL individual pods directly. Clients can connect to a specific pod.

```
# Check DNS from inside cluster:
kubectl run dns-test --rm -it --image=busybox -- nslookup
redis.default.svc.cluster.local
```

Output: three A records — one for each pod's IP (redis-0, redis-1, redis-2)

Q42: Inspect Service Endpoints and explain how they're populated.

```
kubectl get endpoints web-svc
```

→ Endpoints are populated automatically. Kubernetes watches for pods that match the Service's selector label. When a pod with the matching label is Running AND Ready, its IP:port is added to Endpoints. When a pod is deleted or fails its readiness probe, it's removed from Endpoints.

Q43: Demonstrate cross-namespace access using FQDN. Show short name fails.

From namespace 'test', accessing service in namespace 'production':

```
# Short name FAILS from different namespace:
wget web-svc       # fails — only works in same namespace

# FQDN WORKS from any namespace:
wget web-svc.production.svc.cluster.local    # works
```

Q44: Compare kubectl port-forward to a Service versus to a Pod.

Port-forward to a Pod:

```
kubectl port-forward pod/web-abc123 8080:80
```

→ Goes to that specific pod only. If that pod dies, the connection breaks. Use for: debugging one specific pod.

Port-forward to a Service:

```
kubectl port-forward service/web-svc 8080:80
```

→ Routes through the service, which load-balances. If a pod dies, traffic goes to another. Use for: testing your app through the service layer.

Q45: Explain the difference between a Service's port, targetPort, and nodePort.

- **the port clients use to connect to the Service (e.g. 80)**port:
- **the port on the POD/container where the app actually listens (e.g. 8080)**targetPort:
- **the port on the NODE's IP to reach the service from outside (e.g. 30080). Only for NodePort services.**nodePort:

Example: client → Service port 80 → Pod targetPort 8080

Q46: Verify connectivity to ClusterIP Service from throwaway debug pod.

```
kubectl run tmp --rm -it --image=busybox -- sh
# Inside the pod:
wget -qO- http://web-svc
nc -z web-svc 80 && echo 'connected'
```

Q47: Create two Deployments and one Service that load-balances across both by sharing a label.

```
kubectl create deployment app-v1 --image=nginx:1.25
kubectl label deployment app-v1 app=myapp
kubectl label pods -l app=app-v1 app=myapp --overwrite

kubectl create deployment app-v2 --image=httpd:2.4
kubectl label pods -l app=app-v2 app=myapp --overwrite

kubectl create service clusterip myapp-svc --tcp=80:80
```

→ The service routes to all pods with label app=myapp — regardless of which deployment they belong to. Traffic is distributed across both.

Q48: Systematically diagnose why a pod can't reach a Service.

Follow this order:

- 1. DNS — can the pod resolve the service name?

```
kubectl exec POD -- nslookup web-svc
```

- 2. Endpoints — are there any pods registered to the service?

```
kubectl get endpoints web-svc
```

- 3. Selector labels — do pod labels match service selector?

```
kubectl describe service web-svc   # check Selector
kubectl get pods --show-labels
```

- 4. Readiness — are pods passing readiness probes?

```
kubectl describe pod POD_NAME   # check Ready status
```

- 5. Port — is the correct port configured?

```
kubectl describe service web-svc   # check Port and TargetPort
```

Q49: Expose a service with NodePort and verify it works.

```
kubectl expose deployment web --type=NodePort --port=80 --name=web-nodeport
NODE_PORT=$(kubectl get svc web-nodeport -o jsonpath='{.spec.ports[0].nodePort}')
curl http://$(minikube ip):$NODE_PORT
```

Q50: Add HTTP livenessProbe hitting / on port 80 to nginx Deployment.

```
kubectl patch deployment web --patch '
{"spec":{"template":{"spec":{"containers":[{"name":"nginx",
"livenessProbe":{"httpGet":{"path":"/","port":80},
"initialDelaySeconds":5,"periodSeconds":10}}]}}}}'
```

Verify:

```
kubectl describe pod $(kubectl get pods -l app=web -o name | head -1)
```

Look for Liveness section in the output

Q51: Add tcpSocket readiness probe to redis container on port 6379.

```
kubectl patch deployment redis --patch '
{"spec":{"template":{"spec":{"containers":[{"name":"redis",
"readinessProbe":{"tcpSocket":{"port":6379},
"initialDelaySeconds":5,"periodSeconds":5}}]}}}}'
kubectl describe pod -l app=redis   # verify Readiness section
```

Q52: Configure startup probe for ~2-minute boot time.

→ For 2 minutes: failureThreshold * periodSeconds must be >= 120. Example: 24 failures × 5 seconds = 120 seconds.

```
startupProbe:
  httpGet:
    path: /health
    port: 8080
  failureThreshold: 24
  periodSeconds: 5
```

✅ Startup probe buys time for slow-starting apps. Once it passes, liveness probe takes over. Without it, liveness might kill a slow app that's still starting.

Q53: Explain the default behavior when no probes are defined at all.

→ Without probes, Kubernetes assumes the container is ready and healthy as soon as it starts. It never checks if the app is actually working. If the app crashes, Kubernetes restarts it (it detects process exit). But if the app is running but broken (e.g. stuck, not serving requests), Kubernetes has no way to know — traffic keeps going to the broken pod.

LO5 — Solve Problems with Application Shipping Using Containers

For each question: identify the mistake, fix it, and explain what was wrong.

Q1: podman run -d --name web -p 80:8080 nginx — nginx listens on 80, what's reversed?

→ The port mapping format is HOST:CONTAINER. This maps host port 80 to container port 8080. But nginx listens on container port 80, not 8080. The ports are backwards.

Fix:

```
podman run -d --name web -p 80:80 docker.io/library/nginx
```

Or if you want host port 8080:

```
podman run -d --name web -p 8080:80 docker.io/library/nginx
```

Q2: podman run -d --name db -e MYSQL_ROOT_PASSWORD mysql — container exits during init.

→ The -e flag needs a value. MYSQL_ROOT_PASSWORD with no value is invalid. MySQL requires a non-empty root password.

Fix:

```
podman run -d --name db -e MYSQL_ROOT_PASSWORD=mysecretpass
docker.io/library/mysql
```

Q3: podman run -d --name app --network host -p 8080:80 nginx — -p flag is ignored.

→ When you use --network host, the container shares the host's network directly. There is no separate network interface to forward ports to. The -p flag has no effect because port mapping only works when the container has its own network namespace.

Fix: Remove either --network host OR -p, not both.

```
podman run -d --name app -p 8080:80 docker.io/library/nginx
```

Q4: podman run -d --name c1 busybox — goes to Exited (0) immediately. Why?

→ busybox has no default long-running command. It starts, does nothing, and exits immediately with success code 0.

Fix: Give it a command to keep running:

```
podman run -d --name c1 docker.io/library/busybox sleep infinity
```

Q5: podman run --rm -d --name job alpine echo hello — then podman logs job fails.

→ --rm deletes the container as soon as it exits. echo hello finishes instantly, container exits, --rm deletes it. By the time you run podman logs, the container is gone.

Fix: Remove --rm if you need to read logs, or don't use -d (run in foreground):

```
podman run --name job docker.io/library/alpine echo hello
podman logs job
```

Q6: podman run -d -p 8080:80 --memory 8m mysql — database never becomes healthy.

→ 8m = 8 MEGABYTES. MySQL needs at least 256MB-512MB to start. With only 8MB, it runs out of memory and crashes immediately (OOMKilled).

Fix:

```
podman run -d -p 8080:3306 --memory 512m \
  -e MYSQL_ROOT_PASSWORD=pass docker.io/library/mysql
```

Q7: Two alpine containers, exec ping b fails — name resolution fails on default network.

→ On the default bridge network, containers can reach each other by IP but NOT by name. DNS name resolution only works on user-created networks.

Fix: Create a custom network first:

```
podman network create mynet
podman run -d --name a --network mynet docker.io/library/alpine sleep 1d
podman run -d --name b --network mynet docker.io/library/alpine sleep 1d
podman exec a ping b    # now works
```

Q8: podman run -d --name web -v ./html:/usr/share/nginx/html nginx — files not visible or SELinux denies.

→ On SELinux-enabled systems (like RHEL/Fedora), bind mounts are blocked by SELinux by default. You need the :Z or :z flag to relabel the mount.

Fix:

```
podman run -d --name web -v ./html:/usr/share/nginx/html:Z
docker.io/library/nginx
```

:Z = private label (only this container). :z = shared label (multiple containers can access).

Q9: Containerfile/Dockerfile mistakes:

Q10: FROM debian:12 / RUN apt-get install -y nginx — build fails to find package.

→ apt-get install fails because the package lists haven't been downloaded yet. You always need apt-get update before apt-get install.

Broken:

```
FROM debian:12
RUN apt-get install -y nginx
```

Fixed:

```
FROM debian:12
RUN apt-get update && apt-get install -y nginx
```

Q11: FROM python:3.11 / CMD python app.py — can't be stopped cleanly. Shell vs exec form?

→ CMD python app.py uses shell form. This starts a shell (/bin/sh -c) and the shell runs python. When Docker sends SIGTERM to stop the container, the SHELL receives it, not python. Python never gets the signal and can't shut down gracefully.

Broken (shell form):

```
CMD python app.py
```

Fixed (exec form):

```
CMD ["python", "app.py"]
```

✅ Always use exec form (with square brackets) for CMD and ENTRYPOINT. This makes the app the direct process that receives signals.

Q12: FROM node:20 / COPY . /app / RUN npm install — reorder for layer caching.

→ Every time ANY source file changes, Docker invalidates the COPY layer and re-runs npm install from scratch. This is slow. Solution: copy package.json FIRST, run npm install, THEN copy source files. npm install only reruns when package.json changes.

Broken (slow):

```
FROM node:20
COPY . /app
WORKDIR /app
RUN npm install
```

Fixed (fast — uses cache for npm install):

```
FROM node:20
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
```

Q13: FROM alpine:3.20 / ENV PATH=/app/bin / RUN apk add curl — curl not found after.

→ Setting ENV PATH=/app/bin REPLACES the entire PATH variable. The original system PATH (which contains /usr/bin, /bin, etc.) is gone. curl is installed to /usr/bin/curl but /usr/bin is no longer in PATH.

Broken:

```
ENV PATH=/app/bin
```

Fixed (append to existing PATH):

```
ENV PATH=/app/bin:$PATH
```

Q14: FROM ubuntu:24.04 / EXPOSE 8080 / CMD python3 -m http.server 3000 — mapped -p 8080:8080 but nothing answers.

→ EXPOSE 8080 is just documentation — it does NOT make the container listen on that port. The actual server starts on port 3000 (from the CMD). But the port mapping is 8080:8080. There is a mismatch — the server listens on 3000, but you're forwarding to 8080.

Fix option 1 — map to the right port:

```
podman run -p 8080:3000 myimage
```

Fix option 2 — change the server port to match EXPOSE:

```
CMD ["python3", "-m", "http.server", "8080"]
```

✅ EXPOSE tells humans what port the app uses. It does nothing to networking.

Q16: FROM debian:12 / RUN apt-get update && apt-get install -y build-essential — image is huge.

→ apt leaves behind package lists and cache after installing. This wastes space in the image layer.

Fixed (clean up in the same RUN layer):

```
FROM debian:12
```

```
RUN apt-get update && \
    apt-get install -y build-essential && \
    rm -rf /var/lib/apt/lists/*
```

✅ Always clean apt cache in the SAME RUN command. Doing it in a separate RUN doesn't help because the large layer is already created.

Q17: FROM python:3.11 / USER appuser / COPY requirements.txt — permission denied errors.

→ After USER appuser, everything runs as that non-root user. COPY then tries to write files as appuser. But pip install needs to write to system directories that only root can access. The USER directive was set too early.

Broken:

```
FROM python:3.11
USER appuser
COPY requirements.txt /app/
RUN pip install -r /app/requirements.txt
```

Fixed (do root operations first, switch user at the end):

```
FROM python:3.11
COPY requirements.txt /app/
RUN pip install -r /app/requirements.txt
USER appuser    # switch to non-root only for running the app
```

Q18: FROM golang:1.22 / builds app / final image is ~1GB — how does multi-stage build fix it?

→ The golang:1.22 image includes the entire Go compiler, tools, and libraries. You only need those to BUILD the app. The final running binary doesn't need Go at all. Multi-stage build: Stage 1 uses golang to build. Stage 2 uses a tiny image (scratch or alpine) and copies only the binary.

Before (1GB):

```
FROM golang:1.22
COPY . /src
WORKDIR /src
RUN go build -o app .
CMD ["/src/app"]
```

After (tiny — maybe 10MB):

```
# Stage 1: build
FROM golang:1.22 AS builder
COPY . /src
WORKDIR /src
RUN go build -o app .

# Stage 2: run (no Go compiler needed)
FROM alpine:3.20
COPY --from=builder /src/app /app
CMD ["/app"]
```

Q19: A pod is stuck Pending. Walk through diagnosis with kubectl describe pod.

```
kubectl describe pod POD_NAME
```

Look at the Events section at the bottom. Common causes:

- → **node doesn't have enough resources. Fix: lower requests or add nodes.**'Insufficient memory/cpu'
- → **nodeSelector label doesn't match any node. Fix: remove selector or label a node.**'no nodes match nodeSelector'
- → **PVC doesn't exist. Fix: create the PVC first.**'persistentvolumeclaim not found'
- → **general scheduling failure. Read the full message for details.**'Unschedulable'

Q20: Find a deployment, remove spaces, try to deploy it — identify YAML error.

```
kubectl apply -f broken-deployment.yaml
```

→ YAML is indentation-sensitive. Removing spaces breaks the structure. Error messages say something like 'mapping values are not allowed here' or 'unexpected end of stream'. Use a YAML linter to find the problem line.

```
kubectl apply -f broken.yaml --dry-run=client --validate=true
```

✅ Always validate before applying: kubectl apply --dry-run=server -f file.yaml

Q21: A pod is in ImagePullBackOff. Diagnose and fix.

```
kubectl describe pod POD_NAME   # look at Events
```

Common causes:

- → **kubectl set image deployment/web nginx=ngnix:latest (note typo)**Image typo
- → **image:doesnotexist — check available tags**Wrong/missing tag
- → **add imagePullSecrets to pod spec**Private registry without pull secret

Fix image typo:

```
kubectl set image deployment/web nginx=nginx:latest
```

Fix missing pull secret:

```
kubectl create secret docker-registry regcred --docker-server=...
--docker-username=... --docker-password=...
```

Q22: A pod is in CrashLoopBackOff. Use kubectl logs --previous to find root cause.

```
kubectl logs POD_NAME --previous
```

→ --previous shows logs from the LAST crashed instance, not the current one. Read the error messages to find why it crashed. Common causes: missing env vars, can't connect to database, app bug, OOM.

```
kubectl describe pod POD_NAME   # check Last State section
```

Q23: A pod's last state shows OOMKilled. Confirm and fix.

```
kubectl get pod POD_NAME -o yaml | grep -A5 lastState
kubectl describe pod POD_NAME   # look for OOMKilled in Last State
```

→ OOMKilled = Out Of Memory Killed. The container used more memory than its limit. Kubernetes killed it.

Fix option 1 — raise the memory limit:

```
kubectl set resources deployment/NAME --limits=memory=512Mi
```

Fix option 2 — fix the app's memory leak

Q24: A Deployment's pods never become Ready. Trace to failing readiness probe.

```
kubectl describe pod POD_NAME   # look for Readiness probe failed
```

Common causes: wrong path (/health instead of /healthz), wrong port, app not started yet

Fix — adjust the probe to match the app:

```
kubectl set probe deployment/NAME --readiness \
  --get-url http://:8080/healthz \
  --initial-delay-seconds 10
```

Q25: A Service returns nothing. Diagnose Service/pod label mismatch using kubectl get endpoints.

```
kubectl get endpoints web-svc
```

If ENDPOINTS shows <none>, the service can't find any pods.

```
kubectl describe service web-svc   # note the Selector
kubectl get pods --show-labels     # check pod labels
```

→ If service selector is app=web but pods are labeled app=webapp, no pods match. Fix the label.

```
kubectl label pod POD_NAME app=web --overwrite
```

Q26: spec.selector.matchLabels doesn't match spec.template.metadata.labels — explain error.

→ Kubernetes validates that the selector matches the template labels. If they don't match, kubectl apply returns an error: 'selector does not match template labels'. This prevents the deployment from creating pods it can't manage.

Broken:

```
selector:
  matchLabels:
    app: frontend
template:
  metadata:
    labels:
      app: webapp    # doesn't match!
```

Fixed:

```
selector:
  matchLabels:
    app: webapp
template:
  metadata:
    labels:
      app: webapp   # matches selector
```

Q27: resources.requests exceed any node capacity — why Pending, and how to fix.

→ When you set requests: memory: 32Gi but your node only has 16Gi available, the scheduler can't place the pod anywhere. It stays Pending until a node with enough resources is available.

```
kubectl describe pod POD_NAME
```

Events show: 0/1 nodes are available: 1 Insufficient memory

Fix — lower the requests to something the node can satisfy:

```
kubectl set resources deployment/NAME --requests=memory=512Mi
```

Q28: Pod mounting a PVC that doesn't exist — diagnose FailedMount and create the PVC.

```
kubectl describe pod POD_NAME
```

Events show: persistentvolumeclaim 'mypvc' not found

Fix — create the PVC:

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mypvc
spec:
  accessModes: [ReadWriteOnce]
  resources:
    requests:
      storage: 1Gi
---
kubectl apply -f pvc.yaml
```

Q29: Ubuntu container with no long-running command shows CrashLoopBackOff. Explain and fix.

→ Ubuntu (and most base images) have no default command that runs forever. The container starts, the shell exits immediately, and Kubernetes keeps restarting it — CrashLoopBackOff.

Fix — add a long-running command:

```
spec:
  containers:
  - name: app
    image: ubuntu:24.04
    command: ['sleep', 'infinity']
```

Q30: Use kubectl get events to triage everything that happened to a pod.

```
kubectl get events --sort-by=.lastTimestamp
kubectl get events --field-selector involvedObject.name=POD_NAME
kubectl events --for pod/POD_NAME   # newer kubectl versions
```

Q31: Use kubectl exec to get shell and debug DNS with nslookup.

```
kubectl exec -it POD_NAME -- sh
# Inside pod:
cat /etc/resolv.conf      # shows DNS server and search domains
nslookup web-svc          # test service name resolution
nslookup web-svc.default.svc.cluster.local   # full FQDN
```

Q32: Use ephemeral debug container to troubleshoot a distroless/no-shell container.

→ Distroless containers have no shell, no tools. kubectl debug injects a temporary container with tools into the running pod's namespaces.

```
kubectl debug -it POD_NAME --image=busybox --target=CONTAINER_NAME
```

Inside the debug container, you share the process namespace of the target container.

Q33: Spin up throwaway pod to test connectivity to a Service from inside the cluster.

```
kubectl run tmp --rm -it --image=busybox -- sh
# Inside:
wget -qO- http://web-svc
nc -z web-svc 80 && echo OK
```

✅ The --rm flag deletes the pod when you exit the shell. It's a clean way to test without leaving pods behind.

Q34: A node shows NotReady. What to check?

```
kubectl describe node NODE_NAME   # look at Conditions and Events
```

- **is it running? (systemctl status kubelet on the node)**Check kubelet:
- **DiskPressure condition — node is out of disk space**Check disk:
- **MemoryPressure condition — node is out of RAM**Check memory:
- **NetworkUnavailable — CNI plugin issue**Check network:

```
# On minikube:
minikube logs
minikube ssh
```

Q35: A pod is stuck Terminating. Explain finalizers and grace period, then force-delete.

→ Finalizers are cleanup tasks that must complete before a pod is fully deleted (e.g. remove from load balancer). Grace period (default 30s) gives the app time to finish current requests. If either is stuck, the pod stays in Terminating.

```
kubectl delete pod POD_NAME --grace-period=0 --force
```

⚠️ Force delete is risky — the pod might still be running on the node but Kubernetes thinks it's gone. Only use when the node is confirmed dead or the pod is truly stuck.

Q36: Manifest with wrong apiVersion/kind pairing — explain and correct.

→ Each resource type belongs to a specific API group. Pod is in core (v1), Deployment is in apps/v1. Using apps/v1 + Pod is invalid because Pod doesn't exist in the apps group.

Broken:

```
apiVersion: apps/v1
kind: Pod
```

Fixed:

```
apiVersion: v1
kind: Pod
# Deployment:
apiVersion: apps/v1
kind: Deployment
```

Q37: Pod fails with CreateContainerConfigError — configMapKeyRef key is missing.

```
kubectl describe pod POD_NAME   # shows which key is missing
```

→ The pod references a key in a ConfigMap that doesn't exist. Either the key name is wrong or the ConfigMap is missing that key.

Check the ConfigMap:

```
kubectl get configmap my-config -o yaml
```

Fix — either add the missing key or fix the reference in the pod:

```
kubectl create configmap my-config --from-literal=missing-key=value
```

Q38: A pod can't mount a Secret that lives in a different namespace.

→ Secrets (and ConfigMaps, PVCs) are NAMESPACE-SCOPED. A pod in namespace 'dev' CANNOT mount a Secret from namespace 'prod'. This is a fundamental Kubernetes security boundary.

Fix — copy the secret to the same namespace:

```
kubectl get secret my-secret -n prod -o yaml | \
  sed 's/namespace: prod/namespace: dev/' | \
  kubectl apply -f -
```

Q39: Rollout stuck with ProgressDeadlineExceeded.

```
kubectl rollout status deployment/NAME
kubectl describe deployment NAME   # look at Conditions
```

→ ProgressDeadlineExceeded means a rollout didn't complete within the deadline (default 600 seconds). Usually caused by: new pods failing to start, image pull errors, or readiness probe always failing.

Roll back to fix:

```
kubectl rollout undo deployment/NAME
```

Q40: Catch YAML typos before applying — dry-run and validate.

```
kubectl apply --dry-run=client -f deployment.yaml
kubectl apply --dry-run=server -f deployment.yaml
kubectl apply --validate=true -f deployment.yaml
```

✅ dry-run=client catches obvious YAML errors. dry-run=server sends to the API server which validates against the actual schema (catches field typos like imagePullpolicy vs imagePullPolicy).

Q41: App logs say it can't reach database Service. Verify in order.

- → **kubectl get pods -l app=myapp**1. Is the pod running?
- → **kubectl get service db-svc**2. Does the Service exist?
- → **kubectl get endpoints db-svc**3. Are endpoints populated?
- → **kubectl exec POD -- nslookup db-svc**4. Does DNS resolve?
- → **kubectl describe service db-svc (check Port and TargetPort)**5. Is the port correct?

Q42: Read container state/lastState and restartCount with jsonpath.

```
kubectl get pod POD_NAME -o
jsonpath='{.status.containerStatuses[0].restartCount}'
kubectl get pod POD_NAME -o jsonpath='{.status.containerStatuses[0].lastState}'
kubectl get pod POD_NAME -o jsonpath='{.status.containerStatuses[0].state}'
```

Q43: Compare OpenShift Route to Ingress.

Feature	OpenShift Route	Kubernetes Ingress

Where it works	OpenShift only	Any Kubernetes
TLS passthrough	Yes (built-in)	Depends on controller
Traffic splitting	Yes (blue-green)	Limited / controller-dependent
Create command	`oc expose service NAME`	`kubectl create ingress ...`

LO6 — Evaluate Container Orchestration Systems (Theoretical)

Q1: Analyze the networking model of Kubernetes versus Docker's network.

Docker: Each container gets its own IP on a bridge network. Containers talk to each other by name only on user-created networks. Port mapping (-p) is required to expose ports to the host. It's designed for single-host deployments.

Kubernetes: Every pod gets a unique IP across the entire cluster — no NAT between pods. Pods can reach each other directly by IP. Services provide stable DNS names and load balancing. Networking spans multiple nodes via CNI plugins (OVN-Kubernetes, Calico, etc.). Designed for multi-host distributed deployments.

Q2: Evaluate Kubernetes storage abstraction vs Podman's. Which is more flexible for stateful workloads?

Kubernetes: Uses CSI (Container Storage Interface) — a standard that any storage vendor can implement. StorageClasses enable dynamic provisioning (PVCs automatically get a PV). Supports NFS, iSCSI, cloud block storage, local volumes. StatefulSets give each pod its own PVC. Much more flexible for stateful workloads.

Podman: Simpler volume system. Bind mounts or named volumes. No dynamic provisioning. Great for single-host. Not designed for multi-node stateful workloads.

✅ Kubernetes wins for stateful workloads at scale. Podman is simpler for local development.

Q3: Evaluate the operator pattern (CRDs + controllers) for stateful services vs plain manifests.

Plain manifests: You write YAML, you apply it, you manage everything yourself. If your database needs to be backed up, you write a CronJob. If it needs failover, you configure it manually. High effort, full control.

Operator pattern: An Operator is a custom controller that understands your application. It watches for changes and handles day-2 operations automatically (backups, failover, upgrades, scaling). Lower daily effort but more complex to build/trust.

✅ Example: The PostgreSQL Operator can automatically create replicas, do point-in-time recovery, and handle failover — things that would take hundreds of YAML lines to do manually.

Q4: Assess developer experience: plain Kubernetes vs OpenShift S2I and developer console.

Plain Kubernetes: Optimizes for flexibility and control. Developer writes a Dockerfile, builds image, pushes to registry, writes YAML manifests, applies them. Many steps. Steep learning curve. No GUI required but available (k9s, Lens, etc.).

OpenShift S2I: Optimizes for speed and simplicity. Developer points at a Git repo. OpenShift detects the language, builds the image, and deploys it automatically. Web console shows deployments, logs, and metrics visually. Much faster onboarding for developers new to containers.

Q5: Critically evaluate migrating a workload from OpenShift to Kubernetes.

Migration is possible but requires effort:

- Routes must be replaced with Ingress objects

- OpenShift-specific resources (ImageStreams, BuildConfigs) have no equivalent
- S2I builds must be replaced with Dockerfiles + CI/CD pipelines
- Security Context Constraints (SCCs) must be replaced with Pod Security Standards
- Project annotations (UID ranges) work differently

⚠️ Apps that rely heavily on OpenShift-specific features (S2I, Routes, image triggers) require significant rework to run on plain Kubernetes.

Q6: Evaluate running CI/CD build agents inside Kubernetes vs dedicated VMs.

Kubernetes agents: Auto-scale up when builds are needed, scale down to zero when idle. No idle VM costs. Each build gets a clean container. Noisy-neighbor risk if builds consume too many resources. Isolation per pod is good but not VM-level.

Dedicated VMs: Always on (cost even when idle). Easier to manage large build caches. No container overhead. Better isolation. Good for builds that need privileged access.

✅ Kubernetes is better for variable/spiky CI loads. VMs are better for security-sensitive builds or builds that need persistent large caches.

Q7: How does Kubernetes integrate with cloud environments and dynamic capacity provisioning?

- **LoadBalancer services automatically provision cloud LBs (AWS ELB, GCP LB)**Cloud Load Balancers:
- **StorageClasses provision cloud block storage (AWS EBS, GCP PD) on demand**Persistent Volumes:
- **Adds and removes cloud VMs based on pod scheduling pressure**Cluster Autoscaler:
- **Define pools of VMs with different specs (GPU nodes, high-memory nodes)**Node Groups:
- **Pods can assume cloud IAM roles to access S3, GCS, etc.**Cloud Identity:

Q8: Critically evaluate default security: OpenShift vs vanilla Kubernetes.

Kubernetes: Permissive by default — containers can run as root, use host namespaces, mount host paths. Security requires manual configuration (Pod Security Standards, NetworkPolicies, RBAC). Easy to misconfigure.

OpenShift: Restrictive by default — containers get a random non-root UID, SCCs block privileged operations, strict RBAC out of the box, network isolation by default, mandatory TLS everywhere. Much harder to deploy insecure workloads accidentally.

✅ Enterprises in regulated industries (healthcare, finance) choose OpenShift because security is hardened by default, not an afterthought.

Q9: Evaluate the learning curve: OpenShift vs plain Kubernetes.

Plain Kubernetes: Smaller surface area to learn initially (fewer concepts). But you must choose and configure every component (ingress controller, registry, CI/CD, monitoring) yourself. More flexible but more decisions.

OpenShift: More to learn upfront (Routes, Projects, SCCs, image streams, S2I). But once learned, everything is pre-configured and works together. Less configuration, more conventions. Faster for teams that want an 'out of the box' platform.

→ For a team new to containers: OpenShift's conventions help avoid mistakes. For experienced teams that want control: plain Kubernetes gives more flexibility.

Q10: Compare resource footprint: full Kubernetes vs k3s. When is full Kubernetes overkill?

Full Kubernetes: Control plane needs 2-4 nodes (etcd, API server, scheduler, controller manager). Minimum practical cluster: 3-5 nodes. High availability built-in. Thousands of nodes possible.

k3s: Entire control plane runs in a single binary (~100MB). Can run on a Raspberry Pi. Single-node or small clusters. Production-ready but simpler.

→ Full Kubernetes is overkill for: small internal tools, development environments, single-team apps, edge devices. Use k3s or k3d (k3s in Docker) for these cases.

Q11: Small team: Docker Compose on single host vs Kubernetes?

Docker Compose: Simpler. Less learning curve. Fine for one server. No auto-healing, no scaling across nodes, manual updates. Good for: small internal tools, early startup, simple apps.

Kubernetes: Complex setup. Multi-node. Auto-healing, auto-scaling, rolling updates. Good for: growing traffic, high availability requirements, microservices.

→ Recommendation: Start with Docker Compose. Move to Kubernetes when you need: multi-node scaling, zero-downtime deployments, or complex orchestration. Don't over-engineer early.

Q12: Analyze vendor lock-in: Kubernetes (CNCF) vs OpenShift (Red Hat).

Kubernetes: Open source, CNCF project, run by any cloud provider or on-premise. Low lock-in to any vendor. But you choose/manage every component yourself.

OpenShift: Built on top of Kubernetes (so workloads are portable). But OpenShift-specific features (Routes, S2I, ImageStreams) don't exist elsewhere. Migrating away requires rewriting some platform-level config.

→ Kubernetes gives more long-term flexibility. OpenShift gives more features now with some Red Hat dependency. Workloads (your app containers) are portable in both cases.

Q13: Defend recommendation of OpenShift over vanilla Kubernetes for integrated platform.

OpenShift is the better choice when:

- Team wants everything pre-integrated (registry, CI/CD, monitoring, logging)
- Security compliance is required (OpenShift is hardened by default)
- Developer self-service is important (web console, S2I)
- Red Hat support contract is valued
- Team doesn't want to spend time choosing/configuring Kubernetes components

→ OpenShift = Kubernetes + a full platform. Vanilla Kubernetes = just the orchestrator. If you want to avoid building your own platform, choose OpenShift.

Q14: Compare storage provisioning: Kubernetes vs regular VM workloads.

VMs: Storage is provisioned by sysadmin. Mounted as filesystem to OS. Static, manual, managed outside the app.

Kubernetes: PVCs request storage declaratively. StorageClasses provision it dynamically. Storage lifecycle is tied to the workload. More self-service, more automated.

Q15: Startup with 2 engineers must ship quickly and cheaply. Recommend container solution.

→ Recommendation: Docker Compose on a single cloud VM (e.g. DigitalOcean Droplet $12/month).

- Zero Kubernetes complexity
- Simple docker-compose.yml to define all services
- Deploy with docker compose up -d
- Scale vertically (bigger VM) when needed
- Use a managed database (RDS, PlanetScale) to avoid database ops

Move to Kubernetes when: multiple engineers, multiple services, high traffic, or HA requirements appear.

Q16: Compare Docker and Podman: daemon vs daemonless, rootless security.

Docker: Requires a daemon (dockerd) running as root. All docker commands communicate with this root daemon. If the daemon is compromised, attacker gets root. Rootless mode exists but is not default.

Podman: Daemonless — each podman command is its own process. Can run completely rootless by default. Containers run as your user, not root. More secure by default. Compatible with Docker commands (same syntax).

→ Security-conscious teams prefer Podman because: no root daemon to compromise, rootless by default, compatible with Docker workflows.

Q17: Compare day-2 overhead: self-managed Kubernetes vs managed Kubernetes service.

Self-managed (kubeadm, kops): You handle: control plane upgrades, etcd backups, node patching, certificate rotation, scaling nodes, networking plugin updates. High operational burden.

Managed (EKS, GKE, AKS): Cloud provider handles: control plane availability, upgrades, etcd backups. You handle: node upgrades (usually one command), workload deployments. Much lower burden.

✅ For most teams: use managed Kubernetes. Self-managed only if you have specific compliance requirements or need unusual configuration.

Q18: Media streaming company with spiky global traffic — recommend a containerized solution.

→ Recommendation: Managed Kubernetes (EKS/GKE/AKS) with Horizontal Pod Autoscaler + Cluster Autoscaler + CDN.

- HPA scales pods when CPU/traffic spikes
- Cluster Autoscaler adds nodes when pods can't be scheduled
- Multi-region deployments for global latency
- CDN (CloudFront, Fastly) for static content — reduces backend load
- Spot/preemptible nodes for cost savings during scaling

Q19: Company outgrown Docker Swarm — should they migrate to Kubernetes?

Arguments FOR migrating:

- Kubernetes is the industry standard — better ecosystem, more tools, more engineers who know it
- Better autoscaling, health checks, rolling updates
- Managed cloud offerings reduce ops burden

Arguments AGAINST (migration cost):

- Swarm manifests must be rewritten as Kubernetes YAML
- Team must learn Kubernetes (steep curve)
- Risk of migration project taking months

→ Verdict: migrate if you're hitting Swarm's limits (complex networking, multi-cloud, autoscaling). The long-term benefits outweigh the migration cost for any serious production workload.

Q20: Would you choose Kubernetes for microservices architecture?

→ Yes. Kubernetes is ideal for microservices because:

- Each service deploys independently as a separate Deployment
- Services discover each other via DNS (stable service names)
- Each service scales independently based on its own load
- Health probes restart unhealthy services automatically (resilience)
- Rolling updates allow zero-downtime releases per service
- CNCF ecosystem: service mesh (Istio), tracing (Jaeger), metrics (Prometheus)

Q21: Would you choose Kubernetes for enterprise-grade applications?

→ Yes. Kubernetes suits enterprise because:

- **scales to thousands of nodes and tens of thousands of pods**Scalability:
- **largest container orchestration community, CNCF backing, hundreds of contributors**Community:
- **RBAC, network policies, storage abstractions, secrets management, autoscaling**Feature richness:
- **thousands of compatible tools and operators**Ecosystem:
- **enterprise support available from Red Hat, VMware, AWS, Google, Microsoft**Support:

Q22: Would you choose OpenShift for microservices architecture?

→ Yes, especially for enterprise microservices:

- All Kubernetes microservices features included
- Service Mesh (Istio) built-in for traffic management between services
- Developer Console makes it easy to deploy and monitor each service
- Built-in CI/CD (Pipelines / Tekton) for each service's deployment pipeline
- Routes provide zero-config external access per service
- Stricter security between services by default

Q23: Would you choose OpenShift for enterprise-grade applications?

→ Yes, particularly strong case for enterprise:

- Red Hat support contract — 24/7 enterprise support
- Certified operators for common enterprise software (databases, middleware)
- Built-in security hardening — meets compliance requirements out of the box
- Integrated monitoring, logging, and alerting — no third-party setup needed
- Developer self-service — teams deploy without needing platform team for every task
- Consistent across hybrid cloud — same OpenShift on AWS, Azure, GCP, and on-premise

End of Practice Questions — Intro to DevOps